

❖ 栈

要实现表达式求值，首先需要正确理解一个表达式，主要是运算的先后顺序。

[例3.4] 对于算术表达式来说，其基本规则是：

- 先乘除，后加减，先括号内，再括号外；
- 相同优先级情况下从左到右依次计算；
- 比如， $5+6/2-3*4$ 就是一个算术表达式，它的正确理解应该是：
- $5+6/2-3*4 = 5+3-3*4 = 8-3*4 = 8-12 = -4$ 。
- 可以看到这类表达式主要由两类对象构成的，即运算数（如2、3、4等）和运算符（如+、-、*、/等）。
- 不同运算符优先级是不一样的，而且运算符均位于两个运算数中间。

❖ 后缀表达式

- 中缀表达式：运算符位于两个运算数之间。如， $a + b * c - d / e$
- 后缀表达式：运算符位于两个运算数之后。如， $a b c * + d e / -$

【例】 $6\ 2\ /\ 3\ -\ 4\ 2\ *\ +\ =\ ?$ 8



$T(N) = O(N)$ 。不需要知道运算符的优先规则。

❖ 栈的定义

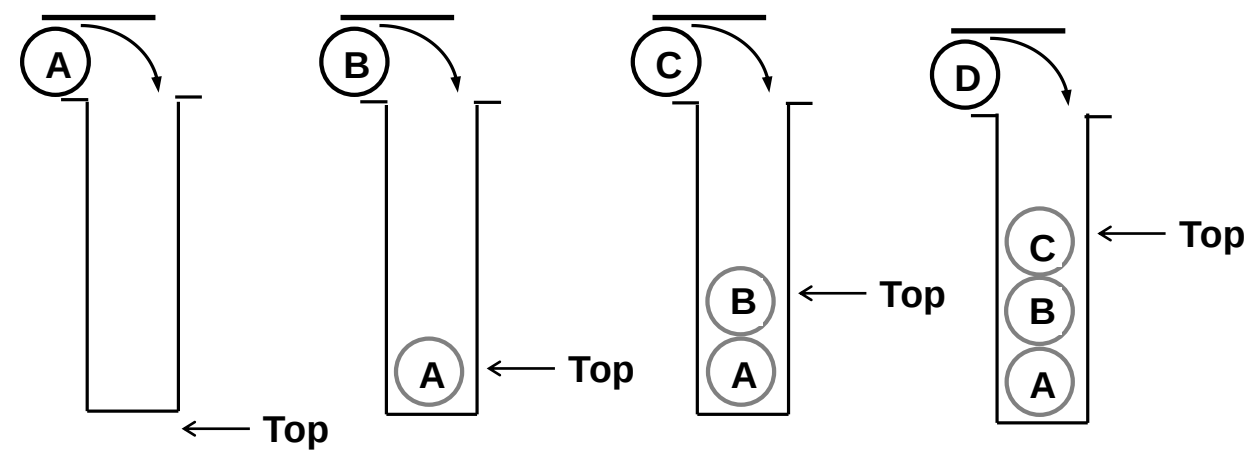
“**栈 (Stack)**”可以认为是具有一定操作约束的线性表，**插入和删除操作**都作用在一个称为**栈顶(Top)**的端点位置。

类型名称: 栈 (Stack)

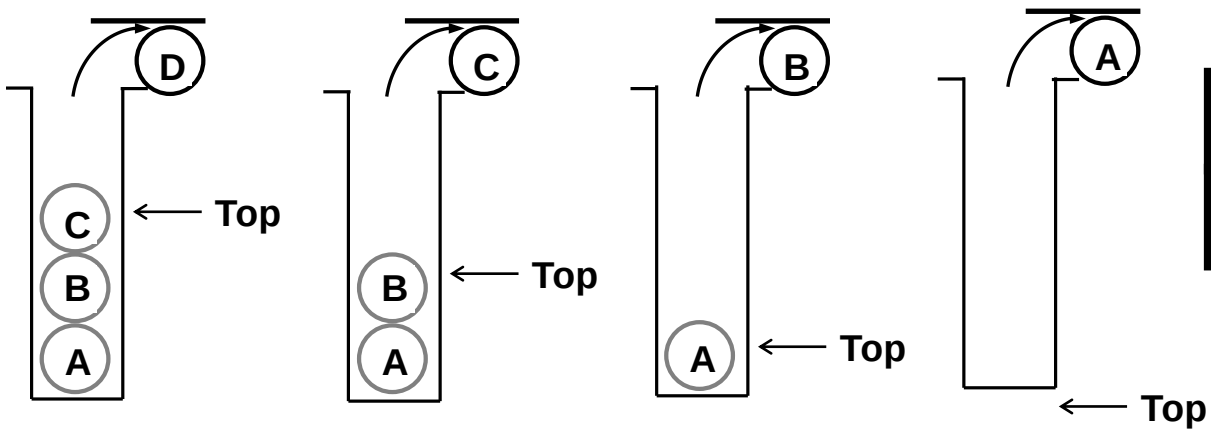
数据对象集: 一个有0个或多个元素的有穷线性表。

操作集: 对于一个具体的长度为正整数MaxSize的栈 $S \in \text{Stack}$ ，记栈中的任一元素 $\text{item} \in \text{ElementType}$ 。

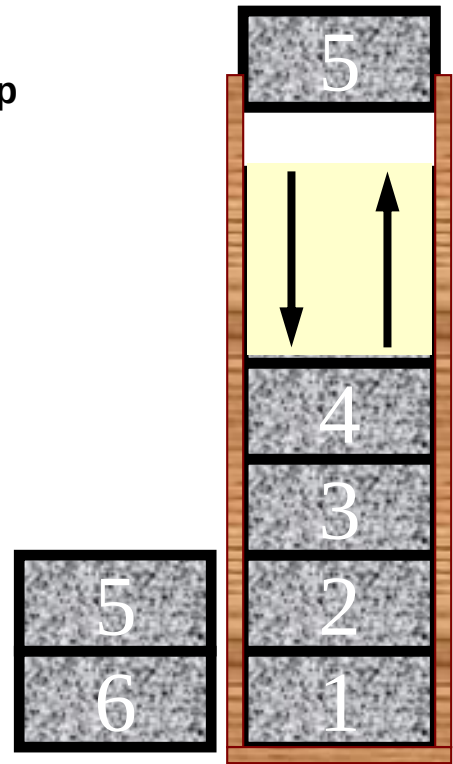
- 1、**Stack CreateStack(int MaxSize)**: 生成空栈，其最大长度为MaxSize;
- 2、**int IsFull(Stack S, int MaxSize)**: 判断栈S是否已满。若S中元素个数等于MaxSize时返回1(TRUE); 否则返回0(FALSE);
- 3、**void Push(Stack S, ElementType item)**: 将元素item压入栈。若栈已满，返回栈为满信息; 否则将数据元素item插入到栈S栈顶处;
- 4、**int IsEmpty (Stack S)**: 判断栈S是否为空，若是返回1(TRUE); 否则返回0(FALSE);
- 5、**ElementType Pop(Stack S)**: 删除并返回栈顶元素。若栈为空，返回栈为空信息; 否则将栈顶数据元素从栈中删除并返回。
- 6、**ElementType Top(Stack S)**:



```
CreatStack( ); Push(S,A); Push(S,B); Push(S,C);
```



```
x=Pop(S); x=Pop(S); x=Pop(S); IsEmpty (S)
```



➤ **Push** 和 **Pop** 可以任意穿插交替进行;

➤ 求后缀表达式 $5\ 6\ 2\ /\ +\ 3\ 4\ *\ -$ 时: **Push** 和 **Pop**的序列是:

```
Push(S,5);
Push(S,6);
Push(S,2); /* S: 5 6 2 */
x=Pop(S); /* x=2 */
y=Pop(S); /* y=6 */
Push(S,y/x); /* y/x=3, S: 5 3 */
x=Pop(S); /* x=3 */
y=Pop(S); /* y=5 */
Push(S,x+y); /* y+x=8 , S: 8 */
Push(S,3);
Push(S,4); /* S: 8 3 4 */
x=Pop(S); /* x=4 */
y=Pop(S); /* y=3 */
Push(S,x*y); /* y*x=12 , S: 8 12 */
x=Pop(S); /* x=12 */
y=Pop(S); /* y=8 */
Push(S,y-x); /* S: -4 */
x=Pop(S); /* x=-4 */
```

[例3.5] 如果将ABCD四个字符按顺序压入栈，是不是ABCD的所有排列都可能是出栈的序列？
可以产生CABD这样的序列吗？

【分析】如果有三个字符ABC出入栈时，全排列有 $3! = 6$ 种可能。

➤ 其中的CAB是没法生成的。

1 因为只有先输入C入栈，C才能从栈中取出，而要求按照ABC这样的顺序进栈，所以C出栈时AB必然还在栈里，而且A还压在B下面。

2 有其它字符排列都可生成，如果进栈顺序为AB，那么出栈的系列AB、BA都有可能：

➤ 即可如果有四个字符ABCD进栈，排列有 $4! = 24$ 种可能，

➤ 也可以是所有排列都有可能是出栈的序列，产生像CABD这样的序列，这两是可能序列也是正好是两个字符的全排列。

➤ 四个字符出栈的所有可能序列有几种？（14种）

1. 栈的顺序存储实现

栈的顺序存储结构通常由一个一维数组和一个记录栈顶元素位置的变量组成。

#define MaxSize <储存数据元素的最大个数>

typedef struct {

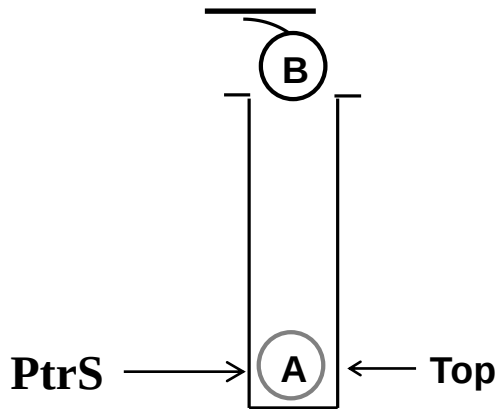
 ElementType Data[MaxSize];

int Top;

} Stack;

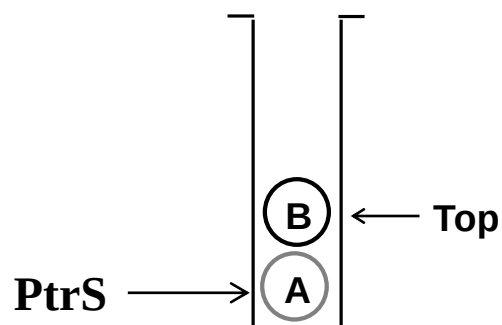
(1)入栈

```
void Push( Stack *PtrS, ElementType
item )
{
    if ( PtrS->Top == MaxSize-1 ) {
        printf("栈满"); return;
    }else {
        PtrS->Data[++(PtrS->Top)] =
item;
        return;
    }
}
```



(2)出栈

```
ElementType Pop( Stack *PtrS )
{
    if ( PtrS->Top == -1 ) {
        printf(“栈空”);
        return ERROR; /* ERROR是ElementType的特殊值，标志错误 */
    } else
        return ( PtrS->Data[(PtrS->Top)--] );
}
```



[例 3.6] 请用一个数组实现两个栈，要求最大地利用数组空间，使数组只要有空间入栈操作就可以成功。写出相应的入栈和出栈操作函数。

【分析】 一种比较聪明的方法是使这两个栈分别从数组的**两头**开始向**中间生长**；当两个栈的**栈顶指针相遇**时，表示两个栈都满了。此时，最大化地利用了数组空间。

```
#define MaxSize <存储数据元素的最大个数>
struct DStack {
    ElementType Data[MaxSize];
    int Top1; /* 栈 1 的栈顶指针 */
    int Top2; /* 栈 2 的栈顶指针 */
} S;

S.Top1 = -1;
S.Top2 = MaxSize;
```

```
void Push( struct DStack *PtrS, ElementType item, int Tag )
{   /* Tag作为区分两个栈的标志, 取值为1和2 */
    if ( PtrS->Top2 - PtrS->Top1 == 1) {   /*栈满*/
        printf("栈满");    return ;
    }
    if ( Tag == 1 ) /* 对第一个栈操作 */
        PtrS->Data[++(PtrS->Top1)] = item;
    else           /* 对第二个栈操作 */
        PtrS->Data[--(PtrS->Top2)] = item;
}

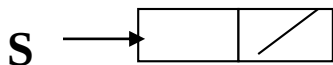
ElementType Pop( struct DStack *PtrS, int Tag )
{   /* Tag作为区分两个栈的标志, 取值为1和2 */
    if ( Tag == 1 ) {   /* 对第一个栈操作 */
        if ( PtrS->Top1 == -1 ) { /*栈1空 */
            printf("栈1空");    return NULL;
        } else return PtrS->Data[(PtrS->Top1)--];
    } else {           /* 对第二个栈操作 */
        if ( PtrS->Top2 == MaxSize ) { /*栈2空 */
            printf("栈2空");    return NULL;
        } else return PtrS->Data[(PtrS->Top2)++];
    }
}
```

2. 栈的链式存储实现

栈的链式存储结构实际上就是一个单链表，叫做链栈。插入和删除操作只能在链栈的栈顶进行；栈顶指针Top就是链表的头指针。

```
typedef struct Node{  
    ElementType Data;  
    struct Node *Next;  
} LinkStack;  
LinkStack *Top;
```

- (1) 栈初始化（建立空栈）
- (2) 判断栈s是否为空



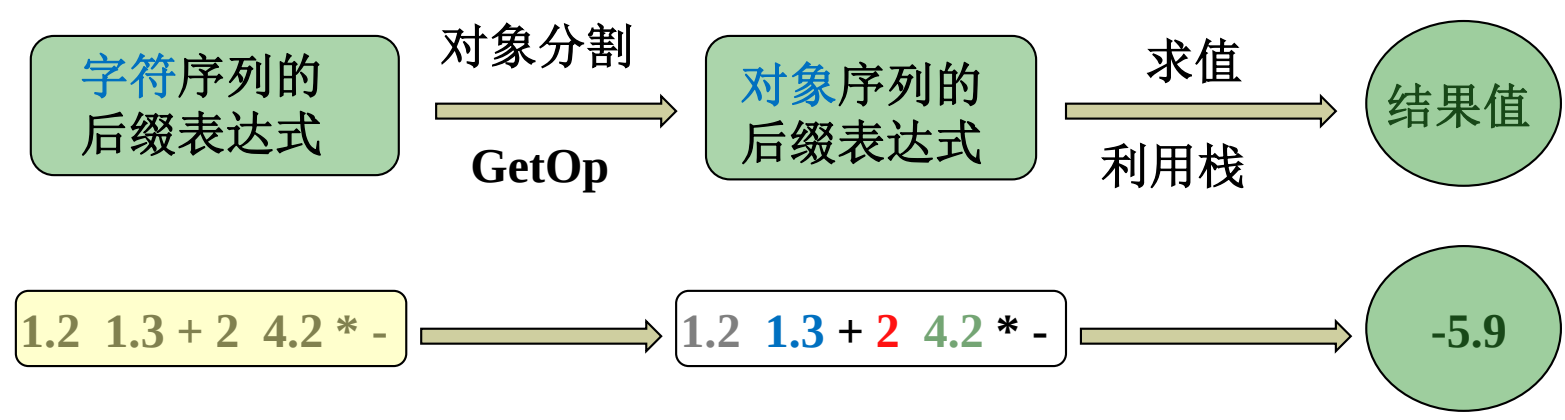
```
LinkStack *CreateStack()  
{ /* 构建一个栈的头结点，返回指针 */  
    LinkStack *S;  
    S = malloc( sizeof( struct Node ) );  
    S->Next = NULL;  
    return S;  
}  
  
int IsEmpty( LinkStack *S )  
{ /*判断栈s是否为空，若为空函数返回整数1，  
   否则返回0 */  
    return ( S->Next == NULL );  
}
```

```
void Push( ElementType item, LinkStack *S )
{   /* 将元素item压入堆栈S   */
    struct Node *TmpCell;
    TmpCell = malloc( sizeof( struct Node ) );
    TmpCell->Element = item;
    TmpCell->Next = S->Next;
    S->Next = TmpCell;
}
```

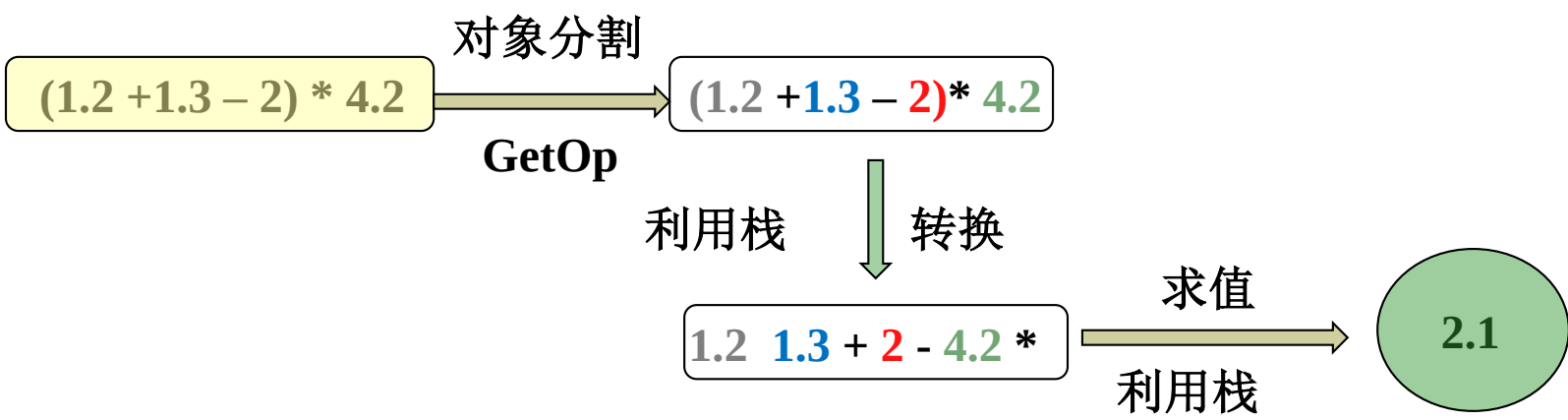
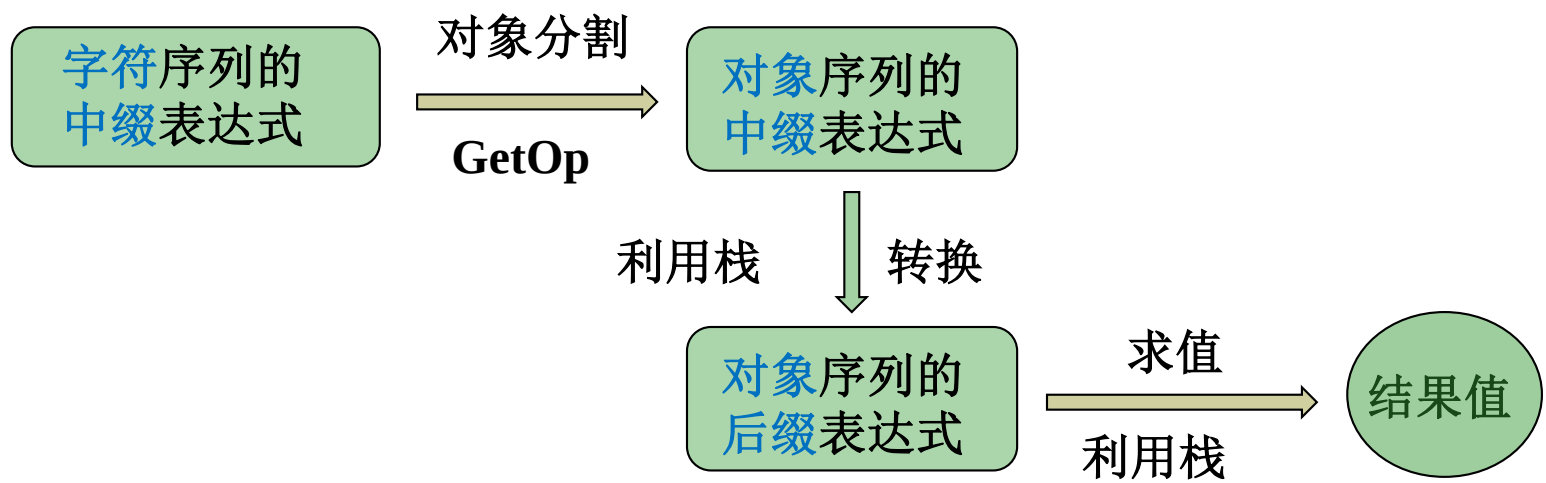
```
ElementType Pop( LinkStack *S )
{   /* 删除并返回堆栈s的栈顶元素 */
    struct Node *FirstCell;
    ElementType TopElem;
    if( IsEmpty( S ) ) {
        printf("栈空"); return NULL;
    } else {
        FirstCell = S->Next;
        S->Next = FirstCell->Next;
        TopElem = FirstCell->Element;
        free(FirstCell);
        return TopElem;
    }
}
```

❖ 栈应用：表达式求值

- 应用栈实现后缀表达式求值的基本过程：
- 从左到右读入后缀表达式的各项（运算符或运算数）；
 - 根据读入的对象（运算符或运算数）判断执行操作；
 - 操作分下列3种情况：
1. 当读入的是一个运算数时，把它被压入栈中；
 2. 当读入的是一个运算符时，就从栈中弹出适当数量的运算数，对该运算进行计算，计算结果再压回到栈中；
 3. 处理完整个后缀表达式之后，栈顶上的元素就是表达式的结果值。



❖ 中缀表达式求值



❖ 中缀表达式转换为后缀表达式

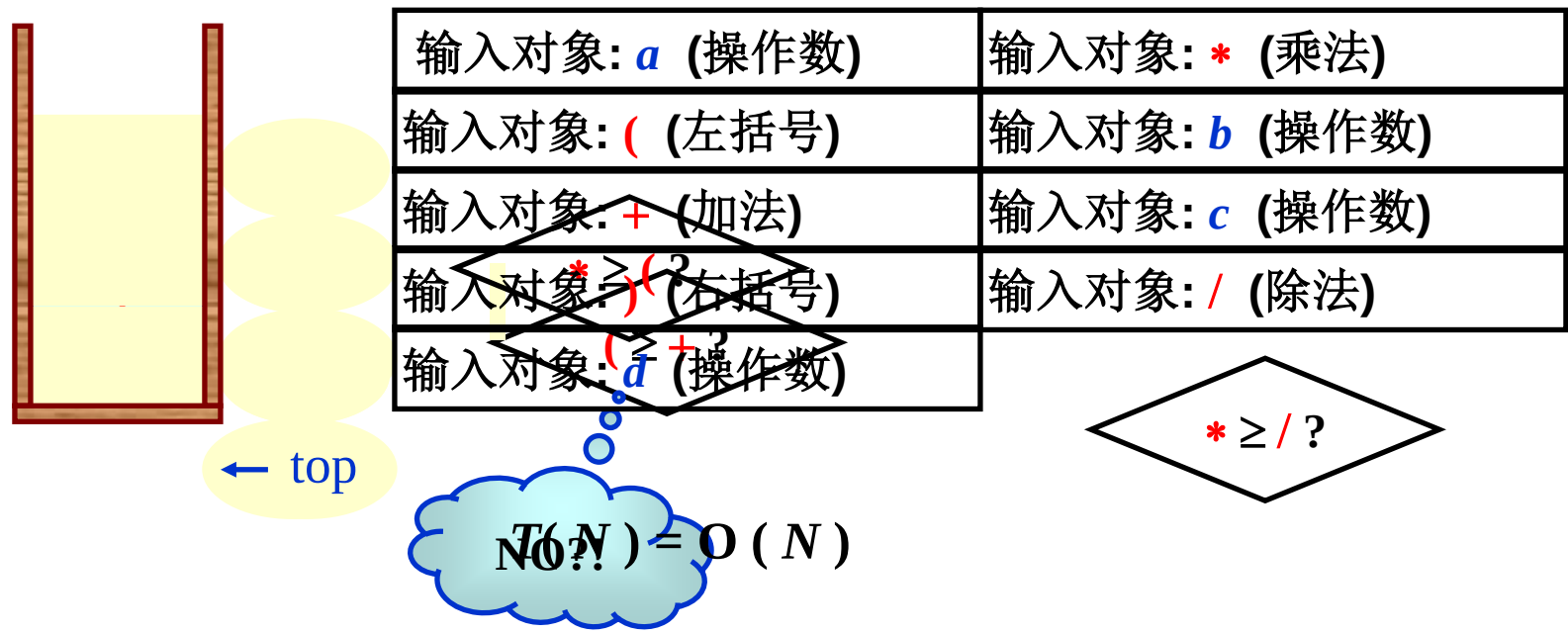
➤ 从头到尾读取中缀表达式的每个对象，对不同对象按不同的情况处理。对象分下列6种情况：

- ① 如果遇到空格则认为分隔符，不需处理；
- ② 若遇到运算数，则直接输出；
- ③ 若是左括号，则将其压入栈中；
- ④ 若遇到的是右括号，表明括号内的中缀表达式已经扫描完毕，将栈顶的运算符弹出并输出，直到遇到左括号（左括号也出栈，但不输出）；
- ⑤ 若遇到的是运算符，若该运算符的优先级大于栈顶运算符的优先级时，则把它压栈；若该运算符的优先级小于等于栈顶运算符时，将栈顶运算符弹出并输出，再比较新的栈顶运算符，按同样处理方法，直到该运算符大于栈顶运算符优先级为止，然后将该运算符压栈；
- ⑥ 若中缀表达式中的各对象处理完毕，则把栈中存留的运算符一并输出。

【例】 $a * (b + c) / d = ?$

$a \ b \ c \ + \ * \ d \ /$

输出: $a \ b \ c \ + \ * \ d \ /$



❖中缀转换为后缀示例： （ 2* （9+6/3-5） +4）

步骤	待处理表达式	栈状态 (底←→顶)	输出状态
1	2* （9+6/3-5） +4		
2	* （9+6/3-5） +4		2
3	（9+6/3-5） +4	*	2
4	9+6/3-5） +4	* （	2
5	+6/3-5） +4	* （	2 9
6	6/3-5） +4	* （ +	2 9
7	/3-5） +4	* （ +	2 9 6
8	3-5） +4	* （ + /	2 9 6
9	-5） +4	* （ + /	2 9 6 3
10	5） +4	* （ -	2 9 6 3 / +
11	） +4	* （ -	2 9 6 3 / + 5
12	+4	*	2 9 6 3 / + 5 -
13	4	+	2 9 6 3 / + 5 - *
14		+	2 9 6 3 / + 5 - * 4
15			2 9 6 3 / + 5 - * 4 +

❖ 队列的定义

“**队列(Queue)**”是具有一定操作约束的线性表，**插入和删除操作**有一定要求：只能在**一端插入**，而在**另一端删除**。

类型名称：队列(Queue)

数据对象集：一个有0个或多个元素的有穷线性表。

操作集：对于一个长度为正整数MaxSize的队列 $Q \in \text{Queue}$ ，记队列中的任一元素 $\text{item} \in \text{ElementType}$ 。

1、**Queue CreatQueue(int MaxSize)**：生成长度为MaxSize的空队列。

2、**int IsFullQ(Queue Q, int MaxSize)**：

判断队列Q是否已满，若是返回1(TRUE)；否则返回0(FALSE)。

3、 **void AddQ(Queue Q, ElementType item)**：

若队列已经满了，返回已满信息；否则将数据元素item插入到队列Q中去。

4、**int IsEmptyQ(Queue Q)**：

判断队列Q是否为空，若是返回1(TRUE)；否则返回0(FALSE)。

5、 **ElementType DeleteQ(Queue Q)**：

若队列为空信息，返回队列空信息；否则将队头数据元素从队列中删除并返回。

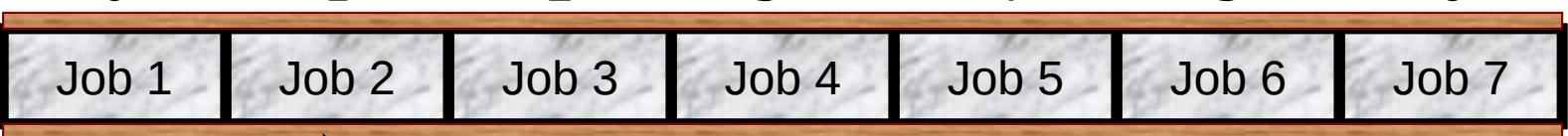
6、**ElementType FrontQ(Queue Q)**：

1. 队列的顺序存储实现

队列的顺序存储结构通常由一个一维数组和一个记录队列头元素位置的变量front以及一个记录队列尾元素位置的变量rear组成。

【例】定义一个大小为MaxSize的队列储存数据元素的最大个数>

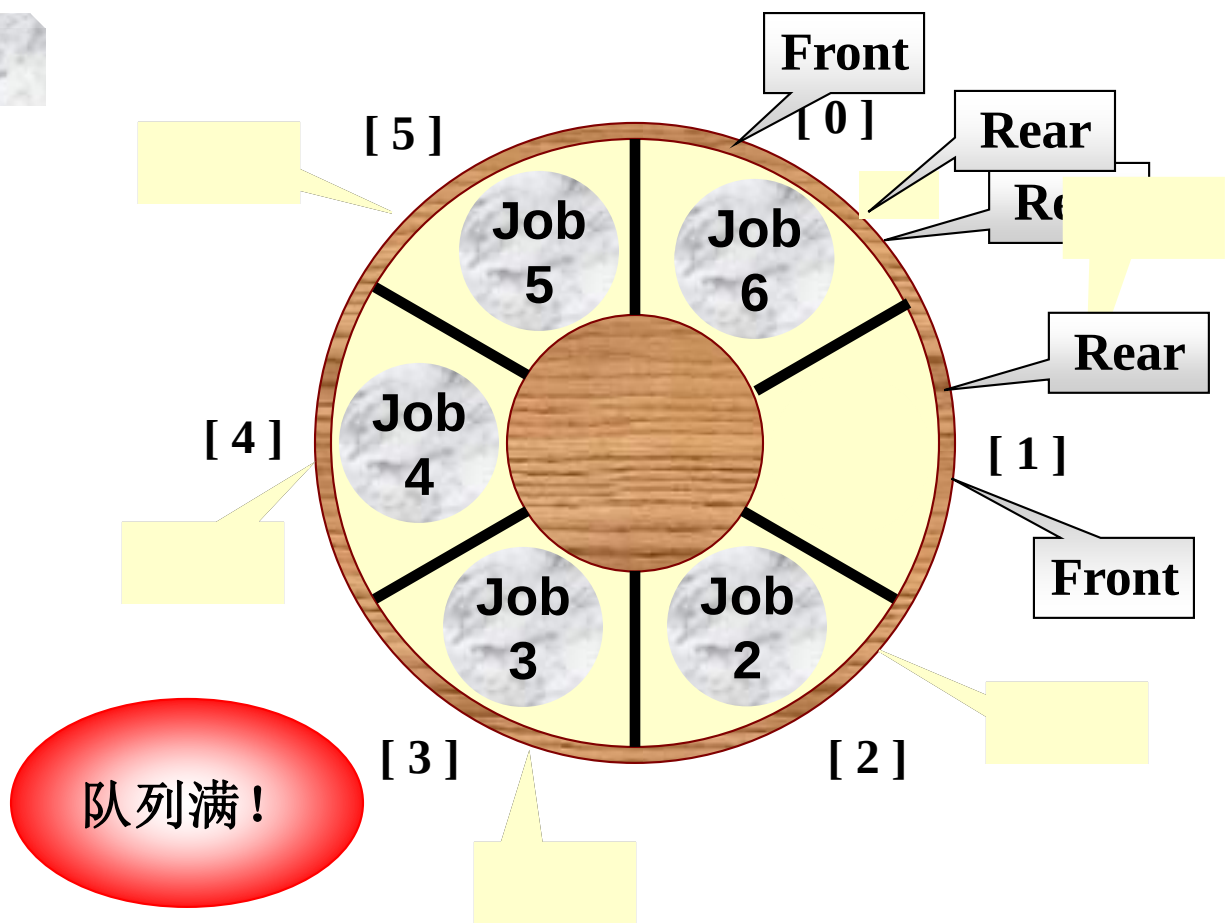
```
typedef struct {
```



AddQ Job 1	AddQ Job 2	AddQ Job 3	DeleteQ Job 1
AddQ Job 4	AddQ Job 5	AddQ Job 6	DeleteQ Job 2
AddQ Job 7	AddQ Job 8		

顺环队列:

- AddQ Job 1
- AddQ Job 2
- AddQ Job 3
- DeleteQ Job 1
- AddQ Job 4
- AddQ Job 5
- AddQ Job 6
- AddQ Job 7



注: 如果数据结构中增加一个 **Size** 域, 用来区分队列“空”和“满”的话, 可以“节省”一个数据元素的存储单元。但是会带来算法描述的复杂性。

(1)入队列

Front和rear
指针的移动
采用“加1取
余”法，体
现了顺序存
储的“循环
使用”。

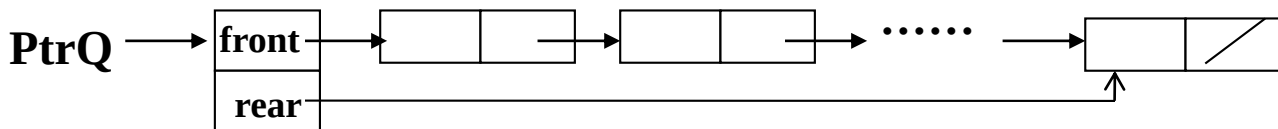
```
void AddQ( Queue *PtrQ, ElementType item)
{
    if ( (PtrQ->rear+1) % MaxSize == PtrQ->front ) {
        printf("队列满");
        return;
    }
    PtrQ->rear = (PtrQ->rear+1)% MaxSize;
    PtrQ->Data[PtrQ->rear] = item;
}
```

```
ElementType DeleteQ ( Queue *PtrQ )
{
    if ( PtrQ->front == PtrQ->rear ) {
        printf("队列空");
        return ERROR;
    } else {
        PtrQ->front = (PtrQ->front+1)% MaxSize;
        return PtrQ->Data[PtrQ->front];
    }
}
```

2. 队列的链式存储实现

队列的链式存储结构实际上就是一个单链表。插入和删除操作在队列的头部和尾部进行。

```
typedef struct Node{  
    ElementType Data;  
    struct Node *Next;  
}QNode;  
typedef struct {    /* 链队列结构 */  
    QNode *rear;    /* 指向队尾结点 */  
    QNode *front;   /* 指向队头结点 */  
} LinkQueue;  
LinkQueue *PtrQ ;
```



❖ 不带头结点的链式队列**出队操作**的一个示例：

```
ElementType DeleteQ ( LinkQueue *PtrQ )
{   Qnode *FrontCell;
    ElementType FrontElem;

    if ( PtrQ->front == NULL ) {
        printf("队列空");   return ERROR;
    }
    FrontCell = PtrQ->front;
    if ( PtrQ->front == PtrQ->rear ) /* 若队列只有一个元素 */
        PtrQ->front = PtrQ->rear = NULL; /* 删除后队列置为空 */
    else
        PtrQ->front = PtrQ->front->Next;
    FrontElem = FrontCell->Data;
    free( FrontCell );          /* 释放被删除结点空间 */
    return FrontElem;
}
```

请学生完成**入队操作**的算法程序。